

程序设计作业：有限元平衡方程组的求解与误差分析

一、作业背景

本次作业对应《2.4 平衡方程组的求解 Solution of Equilibrium Equations》的内容。课件重点介绍了有限元平衡方程组

$$K a = R$$

的直接解法、**LDL^T** 分解、活动列/轮廓存储方法、缓存友好程序实现、多载荷工况求解，以及条件数、残差和舍入误差对计算结果的影响。

本作业是《2.3 系统总体刚度方程》程序设计作业的后续。2.3 作业负责从节点、单元、材料、边界条件和载荷出发，完成对号矩阵、单元刚度矩阵、总体刚度矩阵和缩减方程的建立；本作业负责将 2.3 作业中得到的总体平衡方程交给更专业的线性方程组求解器，并分析求解效率与误差。

请编写程序，实现一个用于对称正定线性方程组的求解器，并将其接入 2.3 作业的有限元程序框架，通过算例分析求解精度、残差、条件数和计算效率。

二、学习目标

完成本作业后，学生应能够：

- 理解有限元平衡方程组中刚度矩阵 **K** 的对称正定特征。
- 实现对称矩阵的 **LDL^T** 分解和前代、对角求解、回代过程。
- 检测零主元或非正主元，并判断矩阵是否适合 **LDL^T** 求解。
- 理解多载荷工况中“先分解矩阵、再求解多个右端项”的效率优势。
- 计算残差向量、相对残差和相对误差。
- 认识病态矩阵中“残差小但解不一定准确”的现象。
- 初步理解活动列/轮廓存储方法对有限元稀疏矩阵求解的意义。
- 学会将有限元总体刚度矩阵转换为稀疏矩阵格式，并调用 MKL/PARDISO/MUMPS 等大规模稀疏方程组求解器。
- 将 2.3 作业中的组装模块与本作业中的求解器模块连接起来，形成完整的有限元计算流程。

三、作业内容

与 2.3 作业的衔接要求

本作业应复用 2.3 作业中已经完成的以下模块：

- 模型输入模块：读取节点坐标、单元连接、材料参数、截面积、边界条件和节点载荷。
- 单元分析模块：计算一维杆单元或二维桁架单元刚度矩阵。
- 对号矩阵模块：生成 **LM**。
- 总体组装模块：由单元刚度矩阵组装总体刚度矩阵 **K**。
- 自由度分块模块：根据已知位移自由度和未知位移自由度生成 **K_FF**、**K_EF**、**d_E**、**f_F**。

本作业主要替换和强化 2.3 作业中的“方程求解”模块。要求将原来的简单求解代码改造成统一接口：

```
def solve_equilibrium(K_FF, rhs, method="ldlt", **options):
    """
    求解缩减平衡方程  $K_{FF} d_F = rhs$ 
    method 可取 "ldlt", "colsol", "mkl_pardiso", "pardiso", "mumps" 等
    """
```

其中：

$$rhs = f_F - K_{EF}^T d_E$$

求得 d_F 后，应回到 2.3 作业的后处理流程，重构完整位移列阵 d ，计算约束反力、单元应力和单元轴力。

如果没有完成 2.3 作业，也可以直接使用本作业给出的矩阵算例；但完整成绩要求能够读取或复用 2.3 作业的桁架模型数据。

任务 1：实现稠密矩阵 LDL^T 求解器

对于对称矩阵 K ，实现分解：

$$K = L D L^T$$

其中 L 为单位下三角矩阵， D 为对角矩阵。

程序至少应包含以下函数：

```
def ldlt_factor(K):
    """
    输入对称矩阵 K
    输出 L, D
    若出现非正主元，应给出错误提示
    """

def ldlt_solve(L, D, R):
    """
    求解  $L D L^T a = R$ 
    包含前代、对角求解和回代
    """

def residual_norm(K, a, R):
    """
    计算残差  $r = R - K a$  及其范数
    """
```

要求：

1. 不允许直接调用 `numpy.linalg.solve`、`scipy.linalg.solve` 等库函数完成本任务的核心求解。
2. 可以使用基础数组和矩阵乘法辅助检查。
3. 分解阶段应检查 $D[j] \leq 0$ 的情况，并输出“矩阵非正定或存在零主元”的提示。

任务 2：误差、残差和条件数分析

构造一个病态线性方程组，例如：

```
K = [[1.0000, 1.0000],
     [1.0000, 1.0001]]

a_exact = [1, 1]^T
R = K a_exact
```

分别使用：

1. 双精度计算。
2. 人为四舍五入到 4 位有效数字的计算。
3. 可选：`float16` 或单精度计算。

比较：

1. 数值解 a 。
2. 残差 $r = R - K a$ 。
3. 相对残差 $\|r\| / \|R\|$ 。
4. 相对误差 $\|a - a_{\text{exact}}\| / \|a_{\text{exact}}\|$ 。
5. 条件数 $\text{cond}(K)$ 。

要求在报告中解释：为什么对于病态问题，残差很小并不一定说明解很准确。

任务 3：大规模稀疏方程组求解器调用

有限元实际问题的自由度数可能达到几十万、几百万甚至更高。此时自行编写的稠密 LDL^T 求解器通常无法承受内存和计算量，需要调用成熟的稀疏直接求解器或并行求解器。

请将大规模有限元方程组存储为稀疏矩阵格式，并调用至少一种外部稀疏求解器完成求解。推荐选择：

1. Intel oneAPI MKL PARDISO。
2. PARDISO Project。
3. MUMPS。

若实验环境允许，鼓励至少比较其中两种求解器。若无法安装多个求解器，至少应完成一种求解器调用，并在报告中说明环境限制。

建议采用以下接口之一：

1. Python: `scipy.sparse` 生成 CSR/CSC 矩阵，调用可用的 MKL/PARDISO/MUMPS Python 绑定或命令行程序。

2. C/C++/Fortran: 直接调用 MKL PARDISO、PARDISO Project 或 MUMPS 官方接口。
3. MATLAB: 调用内置稀疏矩阵求解器, 并选做 MUMPS/PARDISO 接口。

程序应完成:

1. 生成或读取大规模稀疏对称正定矩阵; 对于 Poisson 算例, 矩阵和右端项必须由有限元网格、单元刚度矩阵和单元载荷列阵装配得到。
2. 将矩阵转换为求解器要求的稀疏格式, 例如 CSR、CSC 或坐标格式 COO。
3. 调用外部求解器求解 $K a = R$ 。
4. 输出求解器名称、矩阵阶数、非零元个数、装配时间、求解时间、峰值内存或近似内存占用。
5. 计算残差 $\|R - K a\| / \|R\|$ 。
6. 与自行实现的稠密 LDL^T 求解器在小规模问题上的结果进行对比。

报告中应回答:

1. 为什么大规模有限元矩阵必须使用稀疏存储。
2. CSR/CSC/COO 格式分别如何存储矩阵非零元。
3. 稀疏直接求解器中的符号分解、数值分解和回代分别对应什么工作。
4. 重排序为什么可以减少填充元并降低内存需求。
5. 当矩阵规模继续增大时, 直接求解器可能遇到哪些内存瓶颈。
6. 大规模 Poisson 方程中, 有限元装配时间和线性方程求解时间各占多大比例。

可参考的官方入口:

Intel oneAPI MKL PARDISO:
<https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2024-1/onemkl-pardiso-parallel-direct-sparse-solver-iface.html>

PARDISO Project:
<https://pardiso-project.org/>

MUMPS:
<https://mumps-solver.org/>

四、验证算例

算例 0: 复用 2.3 作业的桁架模型

使用 2.3 作业中的两个验证算例作为本作业的有限元接口测试。

2.3 算例 1: 一维两单元杆结构

从 2.3 作业程序中导出:

```
K = [[ 100, -100,  0],
      [-100,  300, -200],
      [  0, -200,  200]]
```

```
d1 = 0
f2 = 0
f3 = 10
```

要求：

1. 使用本作业的 LDL^T 求解器求解缩减方程。
2. 求得 $d2 = 0.1$, $d3 = 0.15$ 。
3. 回到 2.3 后处理模块，计算节点 1 约束反力和各单元轴力。

2.3 算例 2：二维两杆桁架结构

复用 2.3 作业中的二维两杆桁架输入文件和组装模块。要求：

1. 使用 2.3 作业代码生成 LM 、总体刚度矩阵 K 和缩减矩阵 K_{FF} 。
2. 使用本作业的求解器求解节点 3 位移。
3. 位移结果应接近：

```
u3 = 38.284271
v3 = -10.000000
```

4. 回到 2.3 后处理模块，单元 1 应力应接近 -10.000000 ，单元 2 应力应接近 14.142136 。
5. 在报告中说明：2.3 作业负责“组装方程”，2.4 作业负责“高效求解方程”。

算例 1：三对角对称正定矩阵

构造 $n = 10, 100, 500, 1000$ 的三对角矩阵：

```
K[i,i] = 2
K[i,i-1] = K[i-1,i] = -1
```

取精确解：

```
a_exact = [1, 1, ..., 1]^T
R = K a_exact
```

检查要求：

1. 数值解应接近全 1 向量。
2. 记录不同 n 下的求解时间。
3. 分析计算时间随自由度数 n 增长的趋势。
4. 比较稠密存储和稀疏矩阵的内存需求差异。

算例 2：非正定矩阵检测

测试矩阵：

```
K = [[1, 2],
      [2, 1]]
R = [1, 1]^T
```

检查要求：

1. 程序应检测到非正主元。
2. 程序应停止 LDL^T 求解并给出错误提示。
3. 报告中说明为什么有限元刚度矩阵在缺少足够位移边界条件时可能出现零主元。

算例 4：大规模二维 Poisson 方程有限元求解

求解单位正方形区域上的 Poisson 方程：

```
Omega = (0, 1) x (0, 1)

-Delta u = f,   in Omega
u = 0,         on boundary of Omega
```

采用制造解方法给出理论解：

```
u_exact(x, y) = sin(pi*x) * sin(pi*y)
```

因此右端项为：

```
f(x, y) = 2*pi^2*sin(pi*x)*sin(pi*y)
```

要求使用有限元方法求解，而不是直接构造五点差分矩阵。可选择以下任一种单元：

1. 线性三角形单元 **T3**。
2. 双线性四边形单元 **Q4**。

报告中必须给出有限元理论推导，至少包括：

1. Poisson 方程的强形式。
2. 乘以权函数并分部积分得到的弱形式：
3. 有限元近似：

```
u_h = sum_j N_j a_j
```

4. 单元刚度矩阵和单元载荷列阵：
5. 总体组装、零 Dirichlet 边界条件处理和缩减方程：
6. 所调用大规模求解器的矩阵格式，例如 CSR、CSC 或 COO。

网格规模至少包括：

```
nx = ny = 50
nx = ny = 100
nx = ny = 200
```

其中 nx 和 ny 表示每个方向上的单元数。若采用 Q4 单元，总节点数约为 $(nx+1)*(ny+1)$ ；若采用 T3 单元，可将每个矩形网格剖分为两个三角形。

如果计算机性能允许，可继续测试：

```
nx = ny = 500
nx = ny = 1000
```

程序应输出：

1. 单元类型、节点数、单元数、未知自由度数和总体矩阵非零元个数。
2. 装配时间、边界条件处理时间、求解时间和总时间。
3. 所调用求解器名称，例如 MKL PARDISO、PARDISO Project 或 MUMPS。
4. 相对残差：

$$||R - K a|| / ||R||$$

5. 节点最大误差：

$$\max_i |u_h(x_i, y_i) - u_{\text{exact}}(x_i, y_i)|$$

6. 离散 L2 相对误差，可采用节点近似：

$$\sqrt{\sum_i (u_h_i - u_{\text{exact}_i})^2 / \sum_i u_{\text{exact}_i}^2}$$

7. 至少一张数值解云图或三维曲面图。
8. 至少一张误差云图或误差曲线。

检查要求：

1. 至少调用 MKL/PARDISO/MUMPS 中的一种求解器。

2. 数值解应与理论解 $\sin(\pi x) \sin(\pi y)$ 的形状一致。
3. 网格加密时，节点最大误差和离散 L_2 相对误差应整体下降。
4. 对最小规模，可与自行实现的 LDL^T 或基础求解器结果进行对比。
5. 对更大规模，说明为什么不再适合使用稠密矩阵存储和自行编写的稠密 LDL^T 求解器。
6. 若比较多个求解器，应列出不同求解器的时间、内存、残差和误差。

五、输入与输出要求

建议使用 JSON 文件输入方程组：

```
{
  "Title": "LDLT test",
  "n": 5,
  "K": [[...], [...]],
  "R": [...]
}
```

若输入来自 2.3 作业，建议导出以下格式：

```
{
  "Title": "2.3 truss reduced equation",
  "source_homework": "2-3 Global Stiffness Equations",
  "K_FF": [[...], [...]],
  "rhs": [...],
  "known_dof": [...],
  "free_dof": [...],
  "full_K": [[...], [...]],
  "full_force": [...],
  "known_displacement": [...]
}
```

程序输出至少包括：

1. 矩阵阶数 n 。
2. 是否成功完成 LDL^T 分解。
3. 解向量 $\mathbf{a}$ 。
4. 残差向量 $\mathbf{r}$ 和残差范数。
5. 可选：条件数、计算时间、主元最小值。
6. 对大规模算例，还应输出矩阵非零元个数、稀疏格式、求解器名称和相对残差。
7. 对 2.3 衔接算例，还应输出完整节点位移、约束反力、单元应力和单元轴力。
8. 对 Poisson 方程算例，还应输出理论解、数值解、节点最大误差、离散 L_2 相对误差、数值解图和误差图。

六、报告要求

报告中至少包含：

1. **LDL^T** 分解算法说明。
2. 前代、对角求解、回代流程说明。
3. 与 2.3 作业程序的接口设计：哪些模块复用，哪些模块被本作业替换。
4. 多载荷工况求解效率比较。
5. 活动列/轮廓存储方法说明。
6. 五个验证算例的输入、输出和结果分析。
7. 条件数、残差和解误差之间关系的讨论。
8. 对缓存命中率、半带宽和稀疏矩阵求解效率的简要理解。
9. 大规模稀疏方程组求解器调用过程、运行环境和性能比较。
10. 大规模 Poisson 方程的有限元理论推导、理论解构造、数值解展示和误差收敛分析。

七、提交要求

每位学生提交一个压缩包发到邮箱：yxiao@kust.edu.cn，并上传到github，命名格式为：

学号_姓名_平衡方程组求解程序设计作业.zip

压缩包中应包含：

1. 程序源代码。
2. 至少五个验证算例的输入文件，或能自动生成大规模算例脚本。
3. **README** 运行说明。
4. 作业报告，建议使用 **PDF** 格式。
5. 程序运行结果截图或文本输出。
6. 大规模求解器实验的环境说明，例如 CPU 型号、内存、操作系统、编译器、求解器版本和线程数。
7. 复用 2.3 程序模块的说明。

八、评分标准

1. **LDL^T** 分解实现正确
2. 前代、对角求解、回代实现正确。
3. 非正主元检测和错误处理。
4. 多载荷工况求解实现与效率分析。
5. 活动列/轮廓存储求解器或相应循环范围实现。
6. 大规模稀疏矩阵生成、存储格式转换和外部求解器调用。
7. 残差、相对误差、条件数和大规模 Poisson 方程误差与性能分析。
8. 与 2.3 作业接口衔接清楚，验证算例完整、结果正确。
9. 代码结构、注释、README 和可复现性。

附加题：实现带宽优化或稀疏重排序，例如 Reverse Cuthill-McKee、METIS、AMD 等，并比较优化前后的填充元数量、存储量和求解时间。

九、注意事项

1. 程序中应明确说明数组下标是从 0 还是从 1 开始。
2. 若使用人工智能辅助编程，应在报告中说明辅助内容，并能够解释提交的代码。

3. 任务 3 要求调用 MKL/PARDISO/MUMPS 等外部求解器，但应清楚说明调用接口、矩阵格式和关键参数。
4. 可以使用 NumPy / MATLAB 的基础数组功能，但核心 LDL^T 分解和回代必须自行实现。
5. 计时时应多次运行取平均值，避免单次运行偶然误差。
6. 调用 MKL/PARDISO/MUMPS 时，应在报告中注明安装方式、版本号、线程数和关键参数。
7. 不同机器上的绝对运行时间不可直接比较，重点应比较同一机器、同一问题下不同方法的趋势。